



UNIVERSIDADE FEDERAL DO AMAZONAS - UFAM
FACULDADE DE TECNOLOGIA – FT
PROGRAMA DE PÓS-GRADUAÇÃO EM ENGENHARIA ELÉTRICA - PPGE

GABRIEL NUNES DA SILVA

ANÁLISE COMPUTACIONAL DE PROBLEMAS DE ESCALONAMENTO
MACHINE SCHEDULING, JOB SHOP SCHEDULING E FLOW SHOP

Manaus-AM
Julho-2026

Resumo

Este relatório técnico apresenta uma avaliação computacional de três famílias de problemas de escalonamento: Machine Scheduling, Job Shop Scheduling e Flow Shop Scheduling. A implementação foi organizada em Julia, com JuMP para a construção dos modelos de programação inteira mista e HiGHS como resolvidor principal.

A execução considerada nesta versão utiliza os resultados da rodada inicial do benchmark, com tempo limite de 2 segundos por instância, gap relativo alvo de 1% e configuração registrada $\text{MAX_ITER} = 200$. A análise reúne o valor da função objetivo, o status de término, o gap de otimalidade, o limite dual, o número de nós explorados e o tempo computacional observado.

1. INTRODUÇÃO

Problemas de escalonamento surgem quando é necessário organizar tarefas no tempo, considerando recursos limitados, durações, precedências e possíveis conflitos de utilização. Em aplicações industriais, essa classe de problemas aparece no planejamento de produção, na alocação de máquinas e na redução do tempo total de processamento.

O objetivo deste trabalho é formular e resolver computacionalmente três modelos estudados na disciplina: Machine Scheduling, Job Shop Scheduling e Flow Shop. As instâncias utilizadas foram retiradas das pastas indicadas no enunciado do Trabalho 02, mantendo a ligação com o material da Aula 06 e com a referência do MO-book.

2. MATERIAIS E MÉTODOS

2.1 Ambiente computacional e parâmetros de execução

A implementação foi desenvolvida em Julia, utilizando JuMP como camada de modelagem e HiGHS para resolver os modelos de programação linear inteira mista. O ambiente informado para esta versão é um notebook Acer Aspire 3 A315-33-C39F, com processador Intel Celeron N3060, 4 GB de RAM, HD de 500 GB e Windows 10.

Na rodada de referência empregada na tabela de resultados, adotou-se tempo limite de 2 segundos por instância, gap relativo alvo de 0,01 e parâmetro $\text{MAX_ITER} = 200$ registrado para o controle da execução. Esses limites tornam o ensaio reproduzível e adequado para máquinas com baixa capacidade de processamento.

2.2 Instâncias analisadas

Foram considerados arquivos das pastas `machinescheduling_instances`, `jsplib_subset` e `fssp_problems`. O conjunto de resultados contém 20 execuções, distribuídas em 7 instâncias de Machine Scheduling, 6 de Job Shop e 7 de Flow Shop.

Para cada instância foram armazenados: classe do problema, nome da instância, dimensões principais, valor da função objetivo, status do solver, gap, limite dual, quantidade de nós e tempo de execução.

3. MODELAGEM MATEMÁTICA

3.1 Machine Scheduling

No Machine Scheduling tratado neste relatório, existe uma máquina única e um conjunto de tarefas a serem programadas. Cada tarefa possui tempo de liberação, duração e prazo. A decisão central é determinar os tempos de início de forma que não haja sobreposição entre tarefas e que o atraso total seja minimizado.

O modelo utiliza variáveis contínuas para início, término e atraso, além de variáveis binárias para definir a ordem relativa entre pares de tarefas. A restrição de não sobreposição é representada por desigualdades disjuntivas linearizadas com constante do tipo big-M.

3.2 Job Shop Scheduling

No Job Shop Scheduling, cada job é composto por uma sequência própria de operações. Cada operação deve ser processada em uma máquina específica durante um tempo determinado. O objetivo é minimizar o makespan, isto é, o instante de conclusão do último job.

A formulação combina restrições de precedência dentro de cada job com restrições de capacidade das máquinas. Para operações que disputam a mesma máquina, variáveis binárias indicam qual operação será executada antes, o que eleva a complexidade do modelo.

3.3 Flow Shop Scheduling

No Flow Shop, todos os jobs percorrem as máquinas na mesma ordem. Assim, a principal decisão é encontrar uma permutação de jobs que produza o menor makespan possível.

Foi adotada uma formulação posicional, na qual variáveis binárias associam jobs às posições da sequência. As conclusões em cada máquina são calculadas recursivamente, o que resulta em uma estrutura mais regular que a formulação geral do Job Shop.

4. RESULTADOS COMPUTACIONAIS

A Tabela 1 resume o comportamento das três classes de problemas. Em seguida, são apresentados os resultados detalhados para cada grupo de instâncias. O status "Ótimo" indica que o solver encerrou comprovando a otimalidade segundo os critérios definidos; o status "Tempo limite" indica que o limite de tempo foi atingido antes da prova completa de otimalidade.

4.1 Resumo Por Classe De Problema

Tabela 1 - Síntese das execuções por classe de problema.

Problema	Inst.	Ótimas	Tempo limite	Tempo médio (s)	Gap médio	Maior tempo (s)
Machine Scheduling	7	5	2	0.809	25.71%	2.003
Job Shop	6	1	5	1.711	31.95%	2.009
Flow Shop	7	4	3	1.093	2.85%	2.005

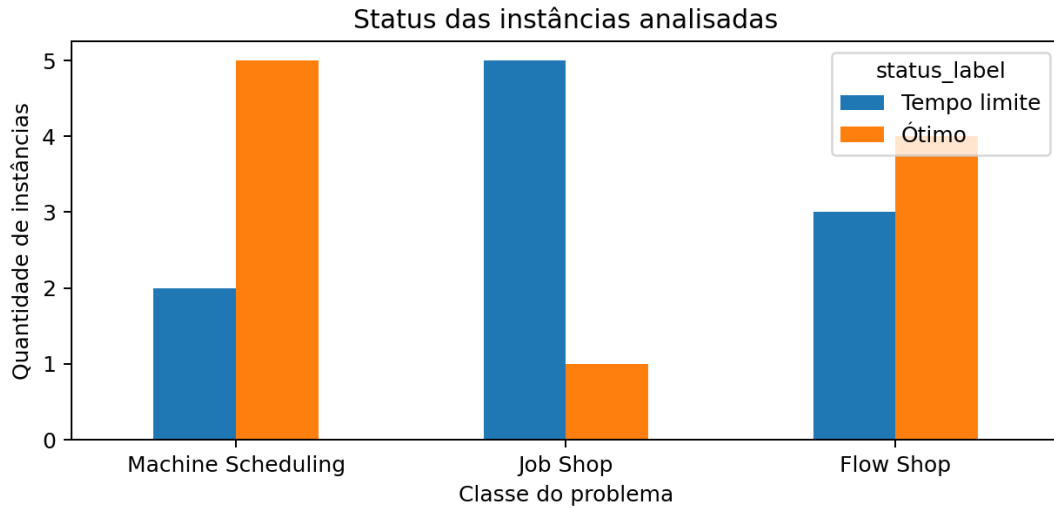


Figura 1 - Distribuição dos status obtidos nas instâncias analisadas.

4.2 Machine Scheduling

Tabela 2 - Resultados das instâncias de Machine Scheduling.

Instância	n	Atraso	Status	Gap	Limite dual	Nós	Tempo (s)
inst_book	7	16	Ótimo	0.00%	16.0	57	0.082
inst_n05_s01	5	7	Ótimo	0.00%	7.0	1	0.007
inst_n07_s01	7	38	Ótimo	0.77%	37.7	48	0.127
inst_n09_s01	9	26	Ótimo	0.01%	26.0	91	0.224
inst_n11_s01	11	16	Ótimo	0.78%	15.9	349	1.216
inst_n13_s01	13	82	Tempo limite	85.41%	12.0	2180	2.003
inst_n15_s01	15	186	Tempo limite	92.99%	13.0	437	2.003

As instâncias de menor dimensão foram resolvidas com rapidez, incluindo inst_book, inst_n05_s01, inst_n07_s01, inst_n09_s01 e inst_n11_s01. Nas instâncias inst_n13_s01 e inst_n15_s01, o solver encontrou soluções viáveis, mas o limite de tempo foi alcançado com gaps elevados, indicando dificuldade crescente conforme aumenta o número de tarefas.

4.3 Job Shop Scheduling

Tabela 3 - Resultados das instâncias de Job Shop Scheduling.

Instância	Jobs	Máq.	Makespan	Ref. ótima	Desvio	Status	Gap	Tempo (s)
ft06	6	6	55	55	-0.00%	Ótimo	0.00%	0.238
la01	10	5	718	666	7.81%	Tempo limite	30.78%	2.003
la02	10	5	820	655	25.19%	Tempo limite	46.00%	2.004

la03	10	5	703	597	17.76%	Tempo limite	40.11%	2.005
la04	10	5	630	590	6.78%	Tempo limite	27.46%	2.009
orb01	10	10	1417	1059	33.81%	Tempo limite	47.37%	2.005

No Job Shop, a instância ft06 foi resolvida até a otimalidade, obtendo makespan igual a 55. As demais instâncias atingiram o limite de tempo, mesmo com soluções viáveis. Isso era esperado, pois a combinação de precedências e conflitos por máquina gera muitas decisões binárias de ordenação.

4.4 Flow Shop Scheduling

Tabela 4 - Resultados das instâncias de Flow Shop.

Instância	Jobs	Máq.	Makespan	Status	Gap	Limite dual	Nós	Tempo (s)
problem_10m_20j	20	10	310	Tempo limite	9.73%	279.8	0	2.005
problem_3m_10j	10	3	126	Ótimo	0.00%	126.0	1	0.004
problem_3m_20j	20	3	218	Ótimo	0.00%	218.0	1	0.012
problem_5m_10j	10	5	129	Ótimo	0.00%	129.0	1	0.127
problem_5m_20j	20	5	265	Tempo limite	3.33%	256.2	0	2.003
problem_5m_30j	30	5	327	Ótimo	0.00%	327.0	1	1.496
problem_8m_10j	10	8	189	Tempo limite	6.92%	175.9	0	2.002

O Flow Shop apresentou desempenho melhor que o Job Shop em diversas instâncias. A estrutura de sequência comum entre as máquinas reduz parte da complexidade da modelagem, o que favoreceu a obtenção de resultados ótimos em problem_3m_10j, problem_3m_20j, problem_5m_10j e problem_5m_30j.

5. DISCUSSÃO

Os resultados evidenciam diferenças importantes entre as três formulações. No Machine Scheduling, a dificuldade aparece principalmente no crescimento do número de pares de tarefas, pois cada par exige uma decisão de ordenação. Mesmo assim, as instâncias pequenas e médias apresentaram bom desempenho.

O Job Shop foi o caso mais exigente. A presença de múltiplas operações por job e a disputa por máquinas diferentes aumentam a quantidade de restrições disjuntivas. Por esse motivo, as instâncias la e orb exigiriam limites de tempo maiores para melhorar a prova de otimalidade.

No Flow Shop, a estrutura de permutação comum torna a formulação mais organizada. Embora algumas instâncias também tenham atingido o limite de tempo, o solver conseguiu provar otimalidade em mais casos quando comparado ao Job Shop.

A configuração com 2 segundos por instância permite uma comparação rápida e controlada. Em computadores com menor capacidade, como notebooks com processador

Celeron e 4 GB de RAM, esse tipo de limite evita execuções longas e facilita a reprodução dos testes.

6. CONCLUSÃO

O trabalho apresentou a implementação e a análise de três problemas clássicos de escalonamento por meio de programação linear inteira mista. Os modelos foram construídos em Julia com JuMP e resolvidos com HiGHS, conforme solicitado no enunciado.

A análise computacional confirmou que o status, o gap, o limite dual e o tempo de execução são informações essenciais para interpretar a qualidade das soluções. Enquanto algumas instâncias foram resolvidas até a otimalidade, outras atingiram o limite de tempo e indicam a necessidade de parâmetros mais amplos para uma investigação mais profunda.

Conclui-se que o HiGHS é adequado para validar as formulações e obter soluções iniciais, especialmente em instâncias pequenas e médias. Para instâncias maiores, recomenda-se aumentar o tempo limite, avaliar outras formulações e comparar diferentes configurações do solver.

REFERÊNCIAS

Bynum, M. L. et al. Hands-On Mathematical Optimization with Python. MO-book, seção Machine Scheduling;
Material da disciplina Otimização - UFAM/PPGEE, Aula 06: escalonamento de máquina, Job Shop e Flow Shop;
JuMP Developers. JuMP: A Modeling Language for Mathematical Optimization;
HiGHS Developers. HiGHS: High Performance Software for Linear Optimization;
JSPLIB - Benchmark instances for the Job Shop Scheduling Problem.

APÊNDICE A - ARQUIVOS E REPRODUÇÃO

O pacote de entrega contém o script `src/trabalho02_benchmark.jl`, as pastas de dados utilizadas, o CSV de resultados de referência e este relatório em PDF.

- Para reproduzir a execução em Julia, instale os pacotes JuMP, HiGHS, CSV, DataFrames, JSON e MathOptInterface. Em seguida, execute o script principal dentro da pasta do trabalho. A versão do Gabriel utiliza a